arXiv:2403.00818v2 [cs.CL] 5 Mar 2024

DenseMamba: State Space Models with Dense Hidden Connection for Efficient Large Language Models

Wei He [*1] **Kai Han** [*1] **Yehui Tang** [1] **Chengcheng Wang** [1] **Yujie Yang** [1] **Tianyu Guo** [1] **Yunhe Wang** [1]

Abstract

Large language models (LLMs) face a daunting challenge due to the excessive computational and memory requirements of the commonly used Transformer architecture. While state space model (SSM) is a new type of foundational network architecture offering lower computational complexity, their performance has yet to fully rival that of Transformers. This paper introduces DenseSSM, a novel approach to enhance the flow of hidden information between layers in SSMs. By selectively integrating shallow-layer hidden states into deeper layers, DenseSSM retains fine-grained information crucial for the final output. Dense connections enhanced DenseSSM still maintains the training parallelizability and inference efficiency. The proposed method can be widely applicable to various SSM types like RetNet and Mamba. With similar model size, DenseSSM achieves significant improvements, exemplified by DenseRetNet outperforming the original RetNet with up to 5% accuracy improvement on public benchmarks. code is avalaible at : https://github.com/WailordHe/DenseSSM.

1. Introduction

Since the release of ChatGPT (OpenAI, 2023), large language models have entered a new epoch, showcasing outstanding abilities in language comprehension, dialogue, and logical reasoning. Over the past year, the industry has witnessed the emergence of numerous large language models, such as LLaMA (Touvron et al., 2023) and ChatGLM (Zeng et al., 2023). These large language models have given rise to a plethora of practical applications, including conversational bots, code assistants, and AI agents. The foundation of large language models lies in the Transformer network structure (Vaswani et al., 2017), primarily utilizing a multi-head self-attention module for modeling relationships between tokens and a Feed-forward network for non-linear feature transformations. The scaling law (Kaplan et al., 2020) based on the Transformer structure has propelled the continuous development and expansion of large language models.

In the Transformer network, multi-head self-attention (MHSA) plays a crucial role, but it comes with significant computational demands and memory requirements during inference. In terms of computational complexity, for an input sentence of length N, the calculation of self-attention has a complexity of $O(N^2)$ during training and inference. Regarding memory usage, previously encountered keys and values are stored, leading to a memory occupation of $O(ND)$. As a result, recent efforts on network architectures have focused on simplifying Transformer by reducing its computation and space complexity. This includes various approaches, notably convolutional language models (Poli et al., 2023), recurrent unit (Lei, 2021), long context models (Ding et al., 2023), and state space models (SSMs) (Gu et al., 2021; Gu & Dao, 2023). These new models have provided strong alternatives to Transformer for building efficient LLMs.

SSMs propose modeling sequences by introducing an appropriate design of hidden states for handling long-range dependencies with both training parallelizability and inference efficiency. Starting from the continuous mapping system, SSMs are discretized to process discrete inputs in deep learning such as language sequence. The discretized SSMs can be computed in both linear recurrence and global convolution modes. Commonly, convolution mode is used during training to achieve parallel acceleration, while recurrence mode is used during autoregressive inference because it has lower computational complexity.

The core distinction of SSMs from other neural networks, such as fully-connected neural networks, lies in the design of hidden states. Hidden states enable information to be propagated along the temporal dimension, while avoiding the computation complexity of accessing historical tokens at each step. Through state transition parameters A, hidden states transfer the hidden information from the previous time

[*]Equal contribution [1]Huawei Noah's Ark Lab. Correspondence to: Kai Han <kai.han@huawei.com>, Yunhe Wang <yunhe.wang@huawei.com>.

steps to the current time step, allowing for autoregressive prediction of the next token. Hidden states play a crucial role in SSMs, but have not received sufficient investigation in the past. Weights and hidden features in different layers contain information at various levels from fine-grained to coarse-grained (Gu et al., 2021). However, in previous versions of SSMs, hidden states only flowed within the current layer and could not transmit more information to deeper layers, thus failing to capture more hierarchical information.

In this paper, we propose DenseSSM to facilitate a more comprehensive flow of hidden information between layers in state space models. We first analyze the hidden state degradation in conventional SSMs which will prevent hidden information flow from low levels to high levels. By selectively integrating shallow-layer hidden states into deeper layers, DenseSSM retains fine-grained information that is useful for the final output. The proposed method is applicable to different types of SSMs, such as RetNet (Sun et al., 2023) and Mamba (Gu & Dao, 2023). Our approach maintains the training parallelizability and inference efficiency of SSMs, while achieving a significant improvement with only a slight increase in the number of parameters. For instance, our DenseRetNet model outperforms traditional RetNet with up to 5% accuracy improvement on public benchmarks.

2. Related Works

2.1. Large Language Models

Large language models (LLMs) have seen transformative advancements, enabling them to excel in a diverse array of natural language processing (NLP) tasks, including machine translation, text summarization, and emergent abilities like incontext learning, which were previously unattainable by earlier language models (Devlin et al., 2019; Raffel et al., 2023). The evolution of LLMs has been marked by a monumental shift in scale, exemplified by models like GPT-3 (Brown et al., 2020), with its 175 billion parameters, and the even more expansive PaLM (Chowdhery et al., 2022), packing in a astounding 540 billion parameters. These models have empirically validated the scaling law (Kaplan et al., 2020), which posits that increasing model size leads to improved performance.

The rapid expansion in model size has underscored the critical need for the development of efficient Transformer algorithms, where FlashAttention (Dao et al., 2022; Dao, 2023) has emerged as a significant innovation. This approach enhances the pivotal attention mechanism within Transformers by optimizing softmax computations using a technique known as tiling. By minimizing memory transactions between the GPU's HBM and on-chip SRAM, FlashAttention compute exact attention with fewer memory accesses, resulting in both faster execution and a lower memory footprint compared to standard attention implementations.

2.2. State Space Models

While the Transformer is currently the de facto architecture for large language models (LLMs), providing efficient parallel GPU training, the inference time for single-token inference increases significantly with longer sequence lengths, posing challenges for deployment due to the O(N) complexity per step even with accelerating algorithms like FlashAttention (Dao et al., 2022; Dao, 2023). Efforts have been dedicated to researching the Transformer-Next architecture, aiming to achieve state-of-the-art (SOTA) performance with efficient parallel training and effective inference, particularly for long sequence lengths.

State Space Sequence Models (SSMs) have recently emerged as promising architectures for sequence modeling. HiPPO (Gu et al., 2020) streamlines sequence modeling by compressing lengthy inputs into a dynamic, polynomial-based representation using orthogonal polynomials. S4 (Gu et al., 2021) introduced a novel parameterization through the application of a low-rank structured correction, enabling stable diagonalization and simplifying the process into Cauchy kernel operations. S5 (Smith et al., 2023) further simplifies the S4 layer by employing a single multi-input, multi-output SSM and introducing efficient parallel scan algorithms into the S4 layers. H3 (Fu et al., 2023) narrows the performance gap between SSMs and Transformer language models by designing three projections (Q, K, V) to simulate the attention mechanism and adopting a fast Fourier transform (FFT) to reduce computation and memory consumption further.

GSS (Mehta et al., 2022) was the first gated neural network architecture incorporating SSMs, it builds upon (Hua et al., 2022) and introducing a compact SSM architecture that contracts model dimensions. Unlike GSS, which emphasizes compressing context into a smaller state, Mamba (Gu & Dao, 2023) diverges by focusing on enhancing the selectivity of the state representation, aiming to balance the tradeoff between efficiency and effectiveness without compromising the model's ability to capture essential information from the context. It achieves this by integrating a selection mechanism which enabling the model to selectively prioritize relevant information while concurrently utilizing a hardware-optimized algorithm that ensures efficient computation.

2.3. Linear Attention

Linear attentions (Katharopoulos et al., 2020; Zhai et al., 2021), which remove the softmax operation from traditional attention, can be seen as a derivative of State Space Models (SSMs). They replace SSMs' convolutions with a variation of Multi-Head Attention (MHA) and eliminate the softmax of the traditional attention mechanism by utilizing a kernel

function that operates independently on the queries (Q) and keys (K). These mechanisms also have a parallel form for efficient training and a recurrent form with $O(1)$ complexity.

RetNet (Sun et al., 2023), TransNormerLLM (Qin et al., 2024), and RWKV (Peng et al., 2023) implement a fixed decay factor to update the previous key-value (KV) states at each recurrent step. This decay mechanism seamlessly integrates with the causal attention mask for efficient parallel computation. However, since this decay factor is preset and independent of the data, it may not be universally applicable across all tasks, especially when prompts or long-range information is particularly important. To address this challenge, GLA (Gated Linear Attention) (Yang et al., 2023) introduces data-dependent gating mechanisms that are practical for both parallel and block-parallel forms. It performs competitively against strong baselines, including the LLaMA-architecture Transformer (Touvron et al., 2023) and Mamba (Gu & Dao, 2023).

3. DenseSSM

In this section, we analyze the hidden state degradation in the deeper layers of SSMs and further introduce dense connection of hidden states to preserve richer information for deeper layers.

3.1. Prelimineries

Transformer Transformer is the widely-used network architecture of large language models which is based on the self-attention mechanism. The self-attention performs as follows:

$$o_t = W_o \frac{\sum_{i=1}^{T} e^{q_t^T k_i} v_i}{\sum_{i=1}^{T} e^{q_t^T k_i}} l, \tag{1}$$

where q, k and v are obtained by fully-connected layers, W_o is the linear transformation weight for the output token o_t at the t-th timestep. Each token will merge information of the other tokens by relationship weights calculated by the self-attention. In addition to self-attention module, the fee-forward network (FFN) module is another key component to transform the token representation and introduces more non-linearity. FFN module is usually composed by two stacked linear layers and non-linear activation function:

$$y_t = W_{down}\sigma(W_{up}o_t), \tag{2}$$

where W_{up} and W_{down} are the weight matrices of up projection and down projection layers, and $\sigma(\cdot)$ is the activation function such as GELU (Hendrycks & Gimpel, 2016).

SSM State space models (SSM) in the literature of deep learning refer to the class of structured SSMs (Gu et al., 2021) and the derivatives such as RWKV (Peng et al., 2023) and RetNet (Sun et al., 2023). Here we briefly describe the structured SSMs as a representative. Structured SSMs define a sequence-to-sequence transformation $x(t) \to y(t)$ with an implicit latent state $h(t)$. The continuous form is formulated as

$$h'(t) = Ah(t) + Bx(t), \tag{3}$$
$$y(t) = Ch(t), \tag{4}$$

where A, B and C are the parameters. To apply SSM to the real discrete data, we discretize the continuous case and obtain the recurrence formulation and convolution formulation of it. The parameters A and B are transformed to the discrete parameters $\overline{A}$ and $\overline{B}$ with the discretization rule such as zero-order hold (Gu et al., 2021). The recurrence formulation is

$$h_t = \overline{A}h_{t-1} + \overline{B}x_t, \tag{5}$$
$$y_t = Ch_t. \tag{6}$$

The convolution formulation is

$$\overline{K} = (C\overline{B}, C\overline{AB}, \cdots, C\overline{A}^t\overline{B}), \tag{7}$$
$$y = x * \overline{K}, \tag{8}$$

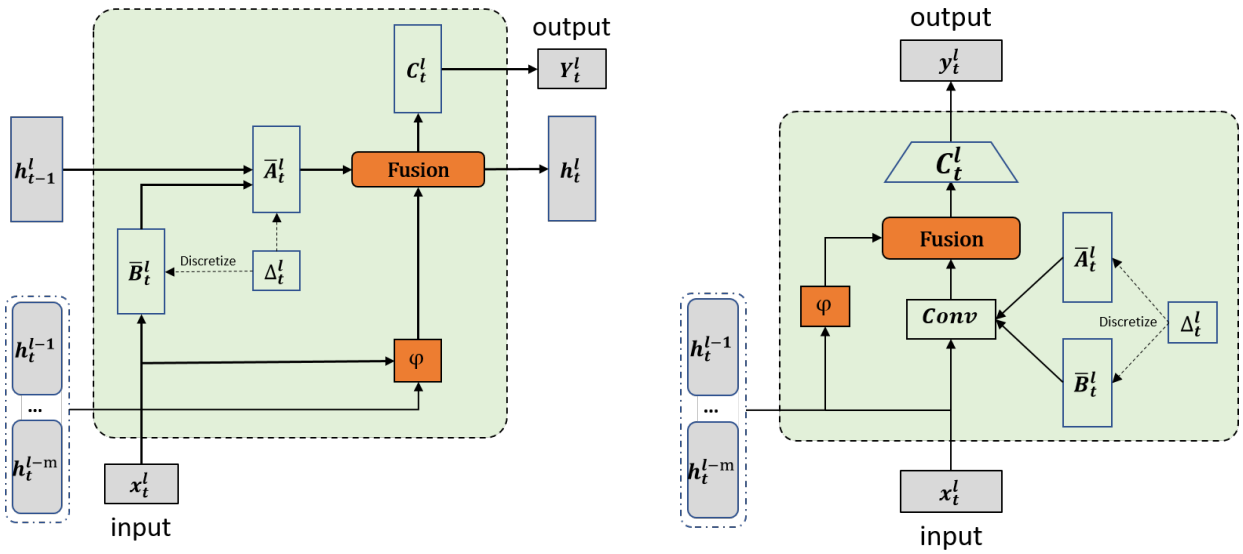
where $*$ is convolution operation, and $t+1$ is the convolution kernel size. The recurrence mode is usually used for efficient autoregressive inference, while the convolution mode is used for efficient parallelizable training.

3.2. Hidden State Degradation

Here we analyze the hidden information flow from shallow layers to deep layers. In the following, we use the superscript "l" to represent the l-th block.

$$\begin{aligned} h_t^l =& \overline{A}h_{t-1}^l + \overline{B}x_t^l \\ =& \overline{A}h_{t-1}^l + \overline{B}\Theta(y_t^{l-1}) \\ =& \overline{A}h_{t-1}^l + \overline{B}\Theta(Ch_t^{l-1}) \\ =& \overline{A}h_{t-1}^l + \overline{B}\Theta(C\overline{A}h_{t-1}^{l-1} + C\overline{B}\Theta(Ch_t^{l-2})) \\ =& \overline{A}h_{t-1}^l + \overline{B}\Theta(C\overline{A}h_{t-1}^{l-1} + \cdots \\ &+ C\overline{B}\Theta(C\overline{A}h_{t-1}^{l-m+1} + C\overline{B}\Theta(Ch_t^{l-m}\underbrace{))\cdots)}_{m}, \end{aligned} \tag{9}$$

where $\Theta(\cdot)$ is the transformations from the last output to the input of SSM module, such as convolution and FFN. From Eq. 9, we can see that the transmission of hidden information from the $(l-m)$-th layer to the l-th layer requires passing through m transformation blocks and m BC matrix multiplications. Such a complex computational process can lead to significant information loss, meaning that attempting to retrieve certain information from the $(l-m)$-th layer at the l-th layer becomes very challenging and unclear.



(a) DenseSSM in autoregressive mode.

(b) DenseSSM in parallelizable convolution mode.

Figure 1. Illustrations of DenseSSM framework, where ϕ is the selective transition module and 'Fusion' is the hidden fusion module.

3.3. Dense Hidden Connection

Through the above analysis, we have identified a crucial issue in SSM, which is the decay of important hidden states as the layer depth increases. Therefore, we propose a dense connection for hidden states to better preserve fine-grained information from shallow layers, enhancing the ability of deep layers to perceive the original textual information. For the l-th block, we densely connect the hidden states in its previous m blocks. First, we collect the shallow hidden states and introduce a selective transition module ϕ to project them to the subspace of the target layer and select useful parts simultaneously:

$$\mathcal{H}_t^l = [\phi(h_t^{l-1}); \phi(h_t^{l-2}); \cdots; \phi(h_t^{l-m})], \tag{10}$$

Then, the intermediate hidden vectors are injected into the original hidden state of this layer:

$$h'^l_t = Fuse(h_t^l, \mathcal{H}_t^l). \tag{11}$$

The operation $Fuse()$ is the function to fuse the intermediate hidden vectors and the current hidden state. The SSMs with the proposed dense hidden connection is named as DenseSSM (Figure 1(a)). The DenseSSM scheme can be used in any SSM variant such as Mamba (Gu & Dao, 2023). Compared to DenseNet (Huang et al., 2017) for convolutional networks, the proposed DenseSSM densely connect the hidden states in SSMs, and the selective mechanism and fusion manner are more efficient for language modeling.

The above analysis is based on the recurrence mode, in the following we introduce the convolution mode of DenseSSM for efficient training. From Eq. 5, we have

$$\begin{aligned} h_t^l &= \overline{A} h_{t-1}^l + \overline{B} x_t^l \\ &= \overline{A}(\overline{A} h_{t-2}^l + \overline{B} x_{t-1}^l) + \overline{B} x_t^l \\ &= \overline{A}^2 h_{t-2}^l + \overline{AB} x_{t-1}^l + \overline{B} x_t^l \\ &= \overline{A}^t h_0^l + \overline{A}^{t-1} \overline{B} x_1^l + \cdots + \overline{AB} x_{t-1}^l + \overline{B} x_t^l \\ &= \overline{A}^t \overline{B} x_0^l + \overline{A}^{t-1} \overline{B} x_1^l + \cdots + \overline{AB} x_{t-1}^l + \overline{B} x_t^l. \end{aligned} \tag{12}$$

This process can be conducted by a convolution on the input sequence $(x_0^l, x_1^l, \cdots, x_t^l)$:

$$\begin{aligned} h_t^l &= \overline{A}^t \overline{B} x_0^l + \overline{A}^{t-1} \overline{B} x_1^l + \cdots + \overline{AB} x_{t-1}^l + \overline{B} x_t^l \\ &= (x_0^l, x_1^l, \cdots, x_t^l) * (\overline{B}, \overline{AB}, \cdots, \overline{A}^t \overline{B}). \end{aligned} \tag{13}$$

In the proposed DenseSSM, we enhance the hidden states by Eq. 11 and then obtain the outputs of SSM:

$$\begin{aligned} y_t^l &= C h'^l_t \\ &= C Fuse((x_0^l, x_1^l, \cdots, x_t^l) * (\overline{B}, \overline{AB}, \cdots, \overline{A}^t \overline{B}), \mathcal{H}_t^l). \end{aligned} \tag{14}$$

As shown in Figure 1(b), DenseSSM can be trained in parallelizable convolution mode.

Selective Transition Module The selective transition module $\phi(\cdot)$ is to project inputs to the target subspace and select the useful part of hidden information simultaneously. We implement the selective transition module with projection layer and gate selection mechanism, as shown in Figure 2. First, we project the hidden states in the previous m

SSM blocks to the same space:

$$h'^{l-m}_t = Proj(h^{l-m}_t). \tag{15}$$

Then we generate the gate weights based on the input x^l_t and use them to select useful hidden states:

$$\phi(h^{l-m}_t) = h'^{l-m}_t \odot Gate(x^l_t). \tag{16}$$

Please note that the newly introduced modules must not compromise the training parallelizability and inference efficiency of the original SSM framework. Therefore, we maintain a simple and efficient implementation in practice. The projection layer is implemented using a linear transformation, while the gate module is implemented with a two-layer MLP with a SiLU activation (Elfwing et al., 2018).

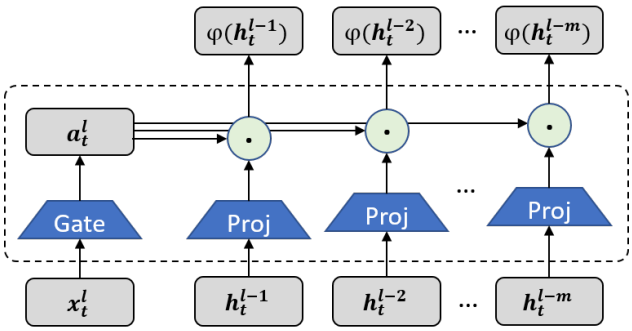


Figure 2. Selective Transition Module.

Hidden Fusion Module After the selective transition module, we obtain the selected hidden states from shallow layers, *i.e.*, $\mathcal{H}^L_t = [\phi(h^1_t); \phi(h^2_t); \cdots; \phi(h^{L-1}_t)]$. A hidden fusion module is utilized to integrate shallow hidden states with the current hidden states. Similarly, we keep the implementation simple for efficiency. We add the selected hidden states since they have been projected to the same space:

$$h^L_t = Fuse(h^L_t, \mathcal{H}^L_t) = h^L_t + \sum_{i=1}^{m} h^{l-i}_t. \tag{17}$$

Here, we provide a basic implementation, but of course, there are other implementation approaches such as concatenation and cross-attention. We will compare different implementation methods in later experiments.

Extension to RetNet RetNet (Sun et al., 2023) can be viewed as a kind of state space models which uses a variant of self-attention rather than convolution in Eq. 7. Compared to the standard Transformer, RetNet is a RNN-style language model with fast inference and parallelized training. It utilizes linear attention to simplify the computation complexity of self-attention.

$$S_t = \gamma S_{t-1} + k^T_t v_t, \tag{18}$$

$$y_t = q_t S_t, \tag{19}$$

where S_t is the recurrent state, and $0 < \gamma < 1$. The dense KV connection for RetNet is performed as follows. The low-level keys and values are first concatenated:

$$\mathcal{K}^l_t = [\phi(k^{l-1}_t); \phi(k^{l-2}_t); \cdots; \phi(k^{l-m}_t)], \tag{20}$$

$$\mathcal{V}^l_t = [\phi(v^{l-1}_t); \phi(v^{l-2}_t); \cdots; \phi(v^{l-m}_t)]. \tag{21}$$

Then, the intermediate key (or value) vectors are injected into the original keys (or values) of this layer:

$$k'^L_t = k^L_t + \sum_{i=1}^{m} k^{l-i}_t, \tag{22}$$

$$v'^L_t = v^L_t + \sum_{i=1}^{m} v^{l-i}_t. \tag{23}$$

The RetNet equiped with the proposed dense key-value (KV) connections is named as DenseRetNet, as illustrated as shown in the figure 3. In addition, the paralleizable mode of DenseRetNet is formulated as follows:

$$y_t = q_t \sum_{i=1}^{t} \gamma^{t-i} k'^T_i v'_i. \tag{24}$$

Our DenseRetNet can be implemented in parallelizable mode as well, that is, can be trained in parallel on GPUs or NPUs.

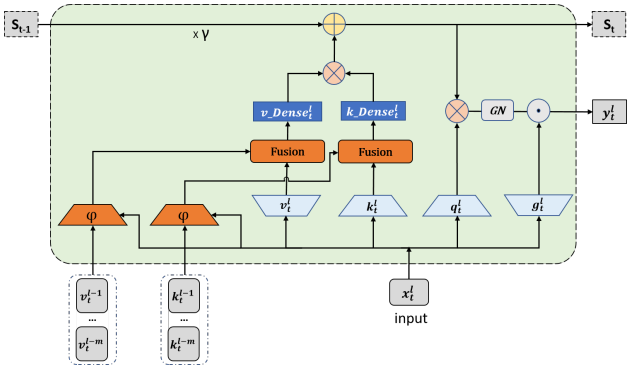


Figure 3. DenseRetNet in autoregressive mode.

4. Experiments

In this section, we conducted comprehensive experiments to validate the effectiveness of the proposed DenseSSM. The verification was carried out on different architectures, including RetNet and Mamba.

4.1. Data and Experimental Settings

Pretraining Data Following the common settings in (Yang et al., 2023), we trained all models from scratch utilizing a corpus comprising 56GB of raw data extracted from The Pile (Gao et al., 2020), a commonly used diverse and high-quality datasets. Excluding data from the DM_Mathematics and Github subsets, we performed a random shuffle and sampled from all remaining corpus. The data was tokenized using the LLaMA tokenizer, which has a vocabulary size of 32,000 tokens. <bos> token was used as the start-of-sequence marker. The resulting cached dataset contained a total of 15 billion tokens.

Evaluation Datasets In our experiment, we investigate models performance across a spectrum of downstream tasks, focusing on zero-shot and 4-shot learning capabilities. The tasks, presented in Table 4 and 6, encompass a range of datasets designed to test common-sense reasoning and question-answering, such as HellaSwag (Zellers et al., 2019), BoolQ (Clark et al., 2019), COPA (Ponti et al., 2020), PIQA (Bisk et al., 2019), Winograd (Muennighoff et al., 2022), Winogrande (Sakaguchi et al., 2019), StoryCloze (Lin et al., 2021), OpenBookQA (Mihaylov et al., 2018), SciQ (Welbl et al., 2017), ARC_E(ARC-easy) and ARC_C(ARC-challenge) (Clark et al., 2018). Words Perplexity results of WikiText (Merity et al., 2016) and LAMBADA (LAMBADA_OPENAI) (Paperno et al., 2016) are also reported. All evaluations are executed using the LM evaluation harness (Gao et al., 2023), ensuring a standardized approach to assessing the models' capabilities.

4.2. Training Setup and Model's Architectures

We selected the 350M and 1.3B model specifications to verify the validity of our proposed dense mechanism. All models were trained from scratch for one epoch on 15 billion tokens. The training batch size was set to 0.5 million tokens with a training length setting of 2048 tokens. AdamW (Loshchilov & Hutter, 2019) optimizer was used for training, with a polynomial learning rate decay, and warm-up ratio is set to 1.5% of total training steps. Weight decay is set to 0.01, and gradient clipping is set to 1. We tailored the hyper-parameters of the model to ensure comparability with models of same scale. Additionally, we designed our Dense RetNet model to be fully comprised of GAU-like blocks, this will be explicitly detailed in the subsequent paragraph.

Transformer-based language models We evaluate our proposed select dense mechanism against popular large language models like LLaMA (Touvron et al., 2023) and OPT (Zhang et al., 2022), comparing with LLaMA for 350M size models and with OPT for 1.3B size models. Table 1 reports their hyperparameters.

Hyperparameters	LLaMA 350M	OPT 1.3B
layers	18	24
hidden size	1024	2048
ffn size	4096	8192
heads	8	32
learning rate	6×10^{-4}	
Adam β	(0.9, 0.98)	
dropout	0.0	0.1

Table 1. Hyperparamters used for LLaMA and OPT models.

Mamba As shwon in Table 2, since our tokenizer is smaller than the GPT-NeoX (Black et al., 2022) tokenzier which Mamba (Gu & Dao, 2023) uses, we have added two additional layers to match the parameters. Besides this, we have adhered to Mamba's model structure and other training settings described in their paper. Specifically, we have set the learning rates to 3e-4 for the 360M model and 2e-4 for the 1.3M model, and we have not applied dropout in either case. The obtained new architecture is named as DenseMamba.

DenseMamba Hyperparameters	360M	1.3B
n_layers	50	50
d_model	1024	2048
dense fusion layers	4	4
learning rate	3×10^{-4}	2×10^{-4}
Adam β	(0.9, 0.95)	
dropout	0.0	

Table 2. Hyperparamters used for DenseMamba models.

RetNet Model sizes and hyperparameters for our DenseRetNet is shown in Table 3. We further utilize Gated Attention Unit (GAU) (Hua et al., 2022) in our DenseRetNet. GAU combine Attention and FFN block into one, so a single block can perform both channel mixing and token mixing: $Y = (XW_u \odot A\hat{V})W_o$, where A is attention weight cauculated though Eq. 24. Also, multiple attention

DenseRetNet Hyperparameters	360M	1.3B
layers	16	25
hidden size	1536	2560
q & k size	768	1280
v & gate size	3072	5120
heads	2	4
dense fusion layers	2	2
learning rate	6×10^{-4}	
Adam β	(0.9, 0.98)	
dropout	0.1	

Table 3. Hyperparamters used for DenseRetNet models.

Models / Tasks	Wikitext	LAMBADA	ARC_C	ARC_E	BoolQ	COPA	HellaSwag	PIQA	WinoGrande	StoryCloze	Winograd	OpenBookQA	SciQ	Avg.
Zero-Shot														
LLaMa-350M	**26.79**	22.50	22.95	**46.13**	**59.27**	64	**33.19**	**64.36**	49.09	57.64	62.02	29.6	75.3	51.23
RetNet-350M	36.88	35.53	21.25	40.99	48.35	61	29.86	62.30	51.07	55.59	59.05	28.4	75.8	48.51
DenseRetNet-350M	31.35	**19.92**	**23.72**	45.03	58.50	**69**	32.31	64.04	**52.09**	**58.04**	**60.82**	**30.4**	**76.6**	**51.87**
Four-Shot														
LLaMa-350M	-	-	23.81	**47.26**	53.00	65	**33.71**	**64.15**	51.14	57.38	**64.25**	**28.2**	**81.2**	**51.73**
RetNet-350M	-	-	23.04	40.91	50.37	63	29.49	62.08	51.78	55.66	59.61	27.4	77.4	49.16
DenseRetNet-350M	-	-	**24.74**	45.66	**54.89**	**69**	32.14	63.70	**52.01**	**57.58**	59.23	**28.2**	78.3	51.41
Zero-Shot														
OPT-1.3B	22.04	13.79	**24.66**	48.65	58.07	**63**	37.00	65.89	**52.80**	**61.02**	65.51	29.6	81.1	53.39
RetNet-1.3B	27.90	23.41	22.61	46.34	48.75	58	32.25	63.44	49.96	57.71	60.65	23.4	77.3	49.13
DenseRetNet-1.3B	**21.55**	**10.88**	24.49	**50.88**	**58.62**	**63**	**38.72**	**67.25**	49.96	60.82	**65.85**	**31.8**	82.7	**54.01**
Four-Shot														
OPT-1.3B	-	-	**25.94**	50.46	52.35	63	36.97	64.64	52.33	60.09	**66.58**	**28.2**	**89.4**	53.63
RetNet-1.3B	-	-	24.66	46.30	47.49	**67**	31.96	63.22	52.09	57.51	61.42	26.6	80.3	50.78
DenseRetNet-1.3B	-	-	25.68	**53.07**	**56.3**	**67**	**38.56**	**66.97**	**53.59**	**62.08**	65.12	27.8	86.7	**54.81**

Table 4. Benchmarking results of DenseRetNet are compared against the original RetNet (Sun et al., 2023) and Transformer-based models, specifically LLaMA-350M (Touvron et al., 2023) and OPT-1.3B (Zhang et al., 2022). Our DenseRetNet architecture has lower perplexity and higher accuracy, effectively enhances the performance of Linear Attention, *e.g.*, in RetNet, and surpasses the performance of Transformer-based Models.

heads with different exponential decay rates are utilized to perform multi-scale decay instead of GAU's single-head strategy. In our experiments, we have observed that our architecture surpasses the RetNet structure with FFN layers in terms of training stability and performance. The obtained new architecture is named as DenseRetNet.

4.3. Main Results for DenseRetNet

We evalute our models on both common corpus, and downstream tasks including common-sense reasoning and question-answering. Table 4 presents the experimental results comparing DenseRetNet with LLaMA-350M (Touvron et al., 2023), OPT-1.3B (Zhang et al., 2022) and RetNet (Sun et al., 2023). Our DenseRetNet obtains lower perplexity on Wikitext and LAMBADA corpus and shows clear advantages in the downstream tasks in both 0-shot and few-shot settings. Especially, our model significantly improves the performance of RetNet, and achieves superior performance compared to the transformer large language models.

4.4. Main Results for DenseMamba

Table 6 compares the performance of DenseMamba with LLaMA-350M (Touvron et al., 2023), OPT-1.3B (Zhang et al., 2022), and Mamba (Gu & Dao, 2023). DenseMamba demonstrates superior perplexity and accuracy on the test set, outperforming Mamba and other Transformer-based models.

4.5. Ablation Studies

In this section, we conduct an ablation study to evaluate the impact of various design choices in our Selective Transition Module and Hidden Fusion Module. Perplexity results are presented for both in-domain evaluation sets and out-of-domain corpora (Merity et al., 2016). For fair comparison, the baseline for all ablation studies is DenseRetNet-350M, with parameter adjustments to facilitate comparisons under similar computational constraints when necessary. We follow the default training settings outlined in Table 3 for our models, except for the model trained on 1B tokens.

Ablations on Selective Transition Module The proposed selective transition module is to project the shallow hidden states to the same subspace and select the useful parts of them. The selective transition module can be implemented in different manners.

Table 5 investigates the impact of various Projection and Select configurations. The experiment's other parameters were held constant: the number of dense layers(m) was set to 2, and the Fusion operation following the selective transition module was an "Add" operation. The findings suggest that the combination of Identity projection with MLP strikes an optimal balance between parameter count and performance.

Projection	Select	#Param	In domain	Wikitext
None	None	346M	2.565	2.359
Identity	MLP	353M	2.546	2.348
Identity	Linear	357M	2.572	2.369
Linear	MLP	353M	2.579	2.372
Linear	Linear	356M	2.582	2.378

Table 5. In-domain evaluation cross-entropy loss and out-of-domain byte_perplexity results for DenseRetNet-350M with various implementations of the selective transition module are presented.

Ablations on Dense Layers In this experiment, we conducted an ablation analysis on the depth of fusion layers (denoted as m) . We employed a fusion strategy based on Identity projection and generate the gate using MLP. Our experimental results, as presented in Table 7, both two-

Models / Tasks	Wikitext	LAMBADA	ARC_C	ARC_E	BoolQ	COPA	HellaSwag	PIQA	WinoGrande	StoryCloze	Winograd	OpenBookQA	SciQ	Avg.
Zero-Shot														
LlaMa-350M	26.79	22.50	22.95	**46.13**	**59.27**	64	33.19	64.36	49.09	57.64	62.02	29.6	75.3	51.23
Mamba-360M	26.60	17.55	23.98	45.83	55.78	61	**34.89**	64.31	**52.88**	58.90	62.92	29.2	**79.8**	51.77
DenseMamba-360M	**26.41**	**17.03**	**24.32**	46.0	59.20	**66**	34.68	**64.80**	51.14	**59.03**	**63.23**	**29.8**	**79.8**	**52.55**
Four-Shot														
LLaMa-350M	-	-	23.81	**47.26**	53.00	65	33.71	64.15	51.14	57.38	64.25	28.2	**81.2**	51.73
Mamba-360M	-	-	**25.26**	46.51	45.41	63	34.25	**65.13**	**52.80**	**58.97**	62.88	**29.0**	81.0	51.29
DenseMamba-360M	-	-	24.83	46.97	**58.26**	**66**	**34.74**	64.69	52.01	58.37	**63.44**	28.6	80.3	**52.56**
Zero-Shot														
OPT-1.3B	22.04	13.79	24.66	48.65	58.07	63	37.00	65.89	**52.80**	61.02	65.51	29.6	**81.1**	53.39
Mamba-1.3B	21.79	**12.46**	**25.09**	50.84	53.15	**67**	38.34	67.19	50.59	60.29	65.25	30.0	79.8	53.41
DenseMamba-1.3B	**21.39**	12.47	**25.09**	**51.89**	**58.59**	**67**	**39.26**	**67.90**	52.01	**61.28**	**66.11**	**30.6**	79.9	**54.51**
Four-Shot														
OPT-1.3B	-	-	25.94	50.46	52.35	63	36.97	64.64	52.33	60.09	**66.58**	28.2	**89.4**	53.63
Mamba-1.3B	-	-	**26.96**	52.69	49.56	**69**	39.25	66.27	52.96	61.15	66.06	30.4	82.3	54.24
DenseMamba-1.3B	-	-	26.54	**52.99**	**58.59**	67	**39.26**	**67.08**	**53.67**	**61.48**	65.89	**31.0**	82.1	**55.05**

Table 6. Benchmarking results of DenseMamba are compared against LLaMA-350M (Touvron et al., 2023), OPT-1.3B (Zhang et al., 2022), and Mamba (Gu & Dao, 2023). The results show that DenseMamba achieves a certain improvement achieving lower perplexity and higher accuracy on the test set compared to Mamba, and surpassing the capabilities of Transformer-based architectures.

layer (m=2) and four-layer (m=4) fusion architectures have performance gains. Considering the computational cost associated with training and inference, the two-layer fusion approach is considered to be more optimal.

In addition, we have explored the necessity of employing distinct gate generation layers for different dense layers. Our experimental results indicate that varying this configuration do not has a positive impact on the model's performance, which is beneficial for the development of lightweight dense connection architectures.

Layers	Diff. gates	#Param	In domain	Wikitext
1	✗	353M	2.570	2.363
2	✗	353M	2.546	2.348
2	✓	360M	2.547	2.351
4	✗	353M	2.542	2.348
4	✓	374M	2.557	2.371

Table 7. In-domain evaluation cross-entropy loss and out-of-domain byte_perplexity results for DenseRetNet-350M with different dense layer numbers and different gate strategies. Diff. gates donates if different gate is applied to different dense features.

Ablations on Hidden Fusion Module The hidden fusion module is to fuse the transited hidden states and the current hidden states. A popular way of feature fusion is by Concat followed by dimensionality reduction, which adds more number of parameters compared to our way. By fine-tuning the model structure, we compare it at the same magnitude, and Table 8 finds that our proposed lightweight Dense hidden connection achieves a better result.

Another study investigates the impact of fusing dense features either every m layers or at each individual layer. To maintain a consistent parameter count, we adjusted the dimensions of the MLP intermediate layer and trained the model with the full 15B tokens. The results in Table 9 indicate that fusing at each layer more effectively facilitates information transfer from lower to higher layers.

Fusion	#Param	In domain	Wikitext
Concat	354M	2.551	2.370
Add	353M	2.546	2.348

Table 8. In-domain evaluation cross-entropy loss and out-of-domain byte_perplexity of DenseRetNet-350M with different implementations of hidden fusion module.

Dense frequency	#Param	In domain	Wikitext
Every layer	353M	2.303	1.845
Every 2 layers	353M	2.331	1.866
Every 4 layers	353M	2.387	1.923

Table 9. In-domain evaluation cross-entropy loss and out-of-domain byte-perplexity for DenseRetNet-350M with varying dense intervention frequency strategies.

5. Conclusion

In this paper, we propose a new DenseSSM framework for enhancing the hidden information flow cross different layers. The hidden states are crucial information storage units in the SSMs. Utilizing the hidden states from each layer more effectively would greatly benefit the fundamental capabilities of SSMs. Therefore, we propose to collect the hidden states from shallow layers and selectively fusing them into the hidden states of deeper layers to enhance the SSM's perception of low-level textual information. The proposed DenseSSM method does not affect the excellent characteristics of SSM, *i.e.*, efficient autoregressive inference and efficient parallelizable training. We apply the DenseSSM method to widely used architectures like RetNet and Mamba, resulting in new architectures with stronger foundational language abilities and achieving higher accuracy in public benchmark evaluations.

6. Impact Statements

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

Bisk, Y., Zellers, R., Bras, R. L., Gao, J., and Choi, Y. Piqa: Reasoning about physical commonsense in natural language, 2019.

Black, S., Biderman, S., Hallahan, E., Anthony, Q., Gao, L., Golding, L., He, H., Leahy, C., McDonell, K., Phang, J., Pieler, M., Prashanth, U. S., Purohit, S., Reynolds, L., Tow, J., Wang, B., and Weinbach, S. Gpt-neox-20b: An open-source autoregressive language model, 2022.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners, 2020.

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., Schuh, P., Shi, K., Tsvyashchenko, S., Maynez, J., Rao, A., Barnes, P., Tay, Y., Shazeer, N., Prabhakaran, V., Reif, E., Du, N., Hutchinson, B., Pope, R., Bradbury, J., Austin, J., Isard, M., Gur-Ari, G., Yin, P., Duke, T., Levskaya, A., Ghemawat, S., Dev, S., Michalewski, H., Garcia, X., Misra, V., Robinson, K., Fedus, L., Zhou, D., Ippolito, D., Luan, D., Lim, H., Zoph, B., Spiridonov, A., Sepassi, R., Dohan, D., Agrawal, S., Omernick, M., Dai, A. M., Pillai, T. S., Pellat, M., Lewkowycz, A., Moreira, E., Child, R., Polozov, O., Lee, K., Zhou, Z., Wang, X., Saeta, B., Diaz, M., Firat, O., Catasta, M., Wei, J., Meier-Hellstern, K., Eck, D., Dean, J., Petrov, S., and Fiedel, N. Palm: Scaling language modeling with pathways, 2022.

Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., and Toutanova, K. Boolq: Exploring the surprising difficulty of natural yes/no questions, 2019.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *ArXiv*, abs/1803.05457, 2018.

Dao, T. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.

Ding, J., Ma, S., Dong, L., Zhang, X., Huang, S., Wang, W., Zheng, N., and Wei, F. Longnet: Scaling transformers to 1,000,000,000 tokens. *arXiv preprint arXiv:2307.02486*, 2023.

Elfwing, S., Uchibe, E., and Doya, K. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks*, 107:3–11, 2018.

Fu, D. Y., Dao, T., Saab, K. K., Thomas, A. W., Rudra, A., and Ré, C. Hungry hungry hippos: Towards language modeling with state space models, 2023.

Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., Presser, S., and Leahy, C. The pile: An 800gb dataset of diverse text for language modeling, 2020.

Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noac'h, A., Li, H., McDonell, K., Muennighoff, N., Ociepa, C., Phang, J., Reynolds, L., Schoelkopf, H., Skowron, A., Sutawika, L., Tang, E., Thite, A., Wang, B., Wang, K., and Zou, A. A framework for few-shot language model evaluation, 12 2023. URL https://zenodo.org/records/10256836.

Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Gu, A., Dao, T., Ermon, S., Rudra, A., and Ré, C. Hippo: Recurrent memory with optimal polynomial projections. *Advances in neural information processing systems*, 33: 1474–1487, 2020.

Gu, A., Goel, K., and Re, C. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2021.

Hendrycks, D. and Gimpel, K. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

Hua, W., Dai, Z., Liu, H., and Le, Q. V. Transformer quality in linear time, 2022.

Huang, G., Liu, Z., Van Der Maaten, L., and Weinberger, K. Q. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4700–4708, 2017.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models, 2020.

Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are rnns: Fast autoregressive transformers with linear attention, 2020.

Lei, T. When attention meets fast recurrence: Training language models with reduced compute. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 7633–7648, 2021.

Lin, X. V., Mihaylov, T., Artetxe, M., Wang, T., Chen, S., Simig, D., Ott, M., Goyal, N., Bhosale, S., Du, J., Pasunuru, R., Shleifer, S., Koura, P. S., Chaudhary, V., O'Horo, B., Wang, J., Zettlemoyer, L., Kozareva, Z., Diab, M. T., Stoyanov, V., and Li, X. Few-shot learning with multilingual language models. *CoRR*, abs/2112.10668, 2021. URL https://arxiv.org/abs/2112.10668.

Loshchilov, I. and Hutter, F. Decoupled weight decay regularization, 2019.

Mehta, H., Gupta, A., Cutkosky, A., and Neyshabur, B. Long range language modeling via gated state spaces, 2022.

Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models, 2016.

Mihaylov, T., Clark, P., Khot, T., and Sabharwal, A. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *EMNLP*, 2018.

Muennighoff, N., Wang, T., Sutawika, L., Roberts, A., Biderman, S., Scao, T. L., Bari, M. S., Shen, S., Yong, Z.-X., Schoelkopf, H., Tang, X., Radev, D., Aji, A. F., Almubarak, K., Albanie, S., Alyafeai, Z., Webson, A., Raff, E., and Raffel, C. Crosslingual generalization through multitask finetuning, 2022.

OpenAI. Chatgpt (mar 14 version). https://chat.openai.com/chat, 2023.

Paperno, D., Kruszewski, G., Lazaridou, A., Pham, Q. N., Bernardi, R., Pezzelle, S., Baroni, M., Boleda, G., and Fernández, R. The lambada dataset, Aug 2016.

Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Cao, H., Cheng, X., Chung, M., Grella, M., GV, K. K., et al. Rwkv: Reinventing rnns for the transformer era. In *Findings of EMNLP 2023*, 2023.

Poli, M., Massaroli, S., Nguyen, E., Fu, D. Y., Dao, T., Baccus, S., Bengio, Y., Ermon, S., and Ré, C. Hyena hierarchy: Towards larger convolutional language models. *arXiv preprint arXiv:2302.10866*, 2023.

Ponti, E. M., Glavaš, G., Majewska, O., Liu, Q., Vulić, I., and Korhonen, A. XCOPA: A multilingual dataset for causal commonsense reasoning. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. URL https://ducdauge.github.io/files/xcopa.pdf.

Qin, Z., Li, D., Sun, W., Sun, W., Shen, X., Han, X., Wei, Y., Lv, B., Luo, X., Qiao, Y., and Zhong, Y. Transnormerllm: A faster and better large language model with improved transnormer, 2024.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer, 2023.

Sakaguchi, K., Bras, R. L., Bhagavatula, C., and Choi, Y. Winogrande: An adversarial winograd schema challenge at scale. *arXiv preprint arXiv:1907.10641*, 2019.

Smith, J. T. H., Warrington, A., and Linderman, S. W. Simplified state space layers for sequence modeling, 2023.

Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., Wang, J., and Wei, F. Retentive network: A successor to transformer for large language models, 2023.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models, 2023.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In *Advances in neural information processing systems*, pp. 5998–6008, 2017.

Welbl, J., Liu, N. F., and Gardner, M. Crowdsourcing multiple choice science questions. In *NUT@EMNLP*, 2017.

Yang, S., Wang, B., Shen, Y., Panda, R., and Kim, Y. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence?, 2019.

Zeng, A., Liu, X., Du, Z., Wang, Z., Lai, H., Ding, M., Yang, Z., Xu, Y., Zheng, W., Xia, X., Tam, W. L., Ma, Z., Xue, Y., Zhai, J., Chen, W., Liu, Z., Zhang, P., Dong, Y., and Tang, J. GLM-130b: An open bilingual pre-trained

model. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=-Aw0rrrPUF.

Zhai, S., Talbott, W., Srivastava, N., Huang, C., Goh, H., Zhang, R., and Susskind, J. An attention free transformer, 2021.

Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X. V., Mihaylov, T., Ott, M., Shleifer, S., Shuster, K., Simig, D., Koura, P. S., Sridhar, A., Wang, T., and Zettlemoyer, L. Opt: Open pre-trained transformer language models, 2022.